AUTOMATIONS 365

The Daily Operation: Knowledge Transfer

Payroll application handover · Evidence-based repository guide

Branch	docs/knowledge-transfer
Commit	9ef7805
Capture date	2026-10-20

Contains system details. Store privately.

How to read this document

[Code] means the statement is supported by this repository and includes a source reference. **[Verify in environment]** means the value or behavior must be checked in Power Platform or on the target workstation.

C/R/U/D/A/S means Create, Read, Update, Delete, Assign, and Share. The action-to-table matrix will use these letters in Part 2.

Phase status

Phase	Title	Status
1	Inventory and setup	Done
2	Data model and action-to-table matrix	Not started
3	Business rules and security	Not started
4	Gaps, risks, and verification	Not started

Contents

Cover	1
How to read this document and summary	2
01 Overview and architecture	4
02 Setup and deployment	7
Part 1 — Inventory and setup	3
Part 2 — Data model and action-to-table matrix	10
Part 3 — Business rules and security	11
Part 4 — Gaps, risks, and verification	12

Application summary

[Code] The Daily Operation is a Microsoft Power Apps Code App built with React, TypeScript, and Vite. Payroll is the current business module. Dataverse is accessed through the Power Apps SDK client and generated services. The current routed areas include payroll administration, employee records, payroll templates, My Payroll, and payslip viewing (`src/App.tsx › App`; `src/router.tsx › router`; `src/lib/power/data-client.ts › dataClient`).

[Verify in environment] User role assignments, app sharing, target environment, Dataverse privileges, and the deployed version must be checked in Power Platform before making an operational change.

PART 1

Inventory and setup

Find the app’s structure, routes, tools, data access seam, and setup/deployment path.

Sections	01 Overview and architecture · 02 Setup and deployment
Status	Done
Questions answered	Where does the app start? What can each role reach? How do I run, build, and publish it?

01 — Overview and architecture

Evidence rule. Every behavior claim below is marked **[Code]** and points to the source that shows it. **[Verify in environment]** means the repository cannot establish the live Power Platform setting. Environment-specific values are intentionally not copied into this handover.

1. What this application is

[Code] The Daily Operation is a React and TypeScript single-page Power Apps Code App. `src/main.tsx` mounts `App`; `src/App.tsx` composes the toast provider, TanStack Query provider, access gate, and router. The Vite build uses the Power Apps Vite plugin (`vite.config.ts`). The app is designed to run in a Power Apps host and also has local development commands (`AGENTS.md`; `package.json › scripts`).

[Code] The code uses the Microsoft Power Apps SDK for app context and Dataverse access. `src/lib/power/power-context.ts › getPowerContext` reads host-provided context; `src/lib/power/data-client.ts › dataClient` binds the SDK data client to generated data-source metadata. Feature services call that client. No hand-written fetch or Axios data layer was found in `src`.

[Code] Dataverse-backed business data is accessed through app services. The repo includes a solution project under `solution/A365_DailyOperations`, generated connector metadata under `.power/schemas`, and generated models and services under `src/generated`. The source tree also contains a Dataverse plug-in project under `plugins/A365.Payroll.Plugins` and helper scripts under `scripts`.

[Verify in environment] The actual environment, deployed app version, solution membership, connections, server-side plug-ins, and role assignments must be checked in Power Platform. `power.config.json` contains an environment binding; its identifier is omitted here because it is environment-sensitive.

2. Repository map

Location	What is there	Evidence
`src/pages/`	Routed screens for payroll, employees, overview, and payslips.	**[Code]** `src/router.tsx › router`
`src/features/payroll/`	Payroll types, status/date rules, UI components, hooks, services, and tests.	**[Code]** `src/features/payroll/`
`src/features/employees/`	Employee UI, user lookup, reference data, hooks, and services.	**[Code]** `src/features/employees/`
`src/components/`	Shared access components and UI primitives.	**[Code]** `src/components/`
`src/providers/`	App-wide TanStack Query and Sonner providers.	**[Code]** `src/App.tsx › App`
`src/lib/power/`	Power Apps SDK context and Dataverse data-client seam.	**[Code]** `src/lib/power/`
`src/generated/`	Generated TypeScript table models and connector services.	**[Code]** `src/generated/index.ts`
`.power/schemas/`	Generated app data-source schemas and registration metadata.	**[Code]** `.power/schemas/appschemas/dataSourcesInfo.ts`
`solution/`	Source project for the Dataverse solution.	**[Code]** `solution/A365_DailyOperations/`
`plugins/`	Source project for the payroll protection plug-in.	**[Code]** `plugins/A365.Payroll.Plugins/`
`scripts/`	Local verification and configuration-seeding scripts.	**[Code]** `scripts/`

Location	What is there	Evidence
`docs/`	Architecture, deployment, payroll, and task documentation.	**[Code]** `docs/`
`public/`	Static logos and app assets.	**[Code]** `public/`

[Code] The browser entry is `index.html` → `src/main.tsx` → `src/App.tsx`. `App` wraps the router with `RoleGate`, `QueryProvider`, and `SonnerProvider`. `src/router.tsx` defines a basename from the current hosted URL and normalizes a hosted `index.html` route for Power Apps.

3. Screens and route access

All routes sit under the global `RoleGate` in `src/App.tsx › App`. The gate allows the two configured app roles or the documented fallback when role lookup is denied. Admin pages also use the `AdminOnly` route wrapper. These are application navigation checks; Dataverse privileges remain the data security boundary (`src/components/RoleGate.tsx › RoleGate`, `src/features/payroll/role-gate.tsx › AdminOnly`, `src/features/payroll/access.ts › ALLOWED_ROLES` and `deriveGate`).

Route	Screen	App-level access	Evidence
`/`	Welcome / role-aware home	Any user admitted by the global gate; page chooses its next action based on access profile.	**[Code]** `src/router.tsx › index route`; `src/pages/home.tsx › HomePage`
`/overview`	Payroll overview	Admin-only route.	**[Code]** `src/router.tsx › AdminOnly`; `src/pages/overview.tsx`
`/employees`	Employee directory	Admin-only route.	**[Code]** `src/router.tsx › AdminOnly`; `src/pages/employees.tsx`
`/payroll`	Payroll periods and dashboard	Admin-only route.	**[Code]** `src/router.tsx › AdminOnly`; `src/pages/payroll.tsx`
`/payroll/templates`	Payroll cutoff templates	Admin-only route.	**[Code]** `src/router.tsx › AdminOnly`; `src/pages/payroll-templates.tsx`
`/payroll/:periodId`	Payroll period detail and entries	Admin-only route.	**[Code]** `src/router.tsx › AdminOnly`; `src/pages/payroll-period.tsx`
`/my-payroll`	Current user's payroll list	Any admitted app role; the query resolves the current user's linked employee and own entries.	**[Code]** `src/router.tsx`; `src/features/payroll/services/my-payroll.ts`
`/my-payroll/:entryId`	Payslip detail	Any admitted app role; service access and Dataverse privileges determine which entry can be read.	**[Code]** `src/router.tsx`; `src/pages/payslip.tsx`; `src/features/payroll/services/my-payroll.ts`

[Code] The navigation shows payroll administration pages to admins and shows My Payroll when the access profile has a linked employee. An admin who also has an employee record can see both sets of links (`src/pages/_layout.tsx › useNav`).

[Verify in environment] The role memberships, app sharing, Dataverse privileges, and whether each user has a linked Employee row cannot be confirmed from the route table. Complete the checks in Part 4's eventual verification checklist.

4. UI, state, and data-fetching patterns

[Code] App-wide server state uses TanStack Query. `QueryProvider` sets queries to no retry, no refetch on window focus, five-minute stale time, and ten-minute garbage collection (`src/providers/query-provider.tsx` › `QueryProvider`). Payroll and employee hooks wrap the corresponding service calls and invalidate query keys after mutations (`src/features/payroll/hooks/`, `src/features/employees/use-employees.ts`).

[Code] `zustand` is listed in `package.json`, but no Zustand store or hook usage was found under `src`. Current data hooks use TanStack Query.

[Code] The UI uses React, React Router, Tailwind CSS, locally provided shadcn-style primitives built on Radix UI, Lucide icons, and Sonner toasts. TanStack Table is used for employee and payroll tables. `date-fns` supports date formatting. `pdfmake` builds downloadable payslip PDFs and is dynamically imported in `src/features/payroll/payslip-document.ts` › `openPayslip`. Recharts is a declared dependency and has a shared chart wrapper in `src/components/ui/chart.tsx`; no business chart usage was found in the page components inspected for this phase.

[Code] The root `README.md` is still the generic starter-template readme; the application-specific orientation is in `docs/`. No app-specific setup section was found in `README.md`.

[Code] Styling starts in `src/index.css` and is processed by the Tailwind Vite plugin configured in `vite.config.ts`. The route shell and visual navigation live in `src/pages/_layout.tsx`.

5. Tests and known documentation gaps

[Code] Vitest runs the test suite (`package.json` › `scripts.test`), with JSDOM and `@testing-library/jest-dom` loaded from `vitest.setup.ts`. The source contains 16 colocated test files: 14 under payroll, one under employees, and one under shared UI. `package.json` has no Playwright script or browser end-to-end test command.

[Code] Not found in code: a SharePoint data source or upload flow for payslips. The payslip code creates a PDF in the client. Do not infer that the PDF is saved to SharePoint (`.power/schemas/appschemas/dataSourcesInfo.ts`, `src/features/payroll/payslip-document.ts`).

[Verify in environment] Not found in code: a confirmed current deployment target, production URL, active connection owner, actual role configuration, or published app release status. The repository contains a deployment runbook, but its live-environment statements must be rechecked before use (`docs/deployment.md`).

02 — Setup and deployment

Evidence rule. [Code] statements describe files or behavior visible in this repository. **[Verify in environment]** statements require a current Power Platform or workstation check. This guide omits connection strings, credentials, environment IDs, organization URLs, app IDs, and solution GUIDs.

1. Prerequisites

[Verify in environment] Microsoft lists an IDE, Node.js LTS, npm, Git, and Power Apps CLI as prerequisites for Power Apps Code Apps. The target Power Platform environment must also have Code Apps enabled. Confirm these before setup in the current workspace and admin center. Microsoft Learn: Code Apps overview.

[Code] This repository has a lockfile and uses npm scripts. Its manifest declares `@microsoft/power-apps-cli` version `1.0.1` as a development dependency. The current inspection workstation reported Node `v24.21.0`, npm `11.19.0`, and local `pa` CLI `1.0.1`; these are capture-time values, not a version policy (`package.json`, `package-lock.json`; `node --version`, `npm --version`, and local `pa --version`).

2. First-day local checklist

1. **[Verify in environment]** Obtain the repository through the team's approved

source-control process. Start in the repository root.

1. **[Verify in environment]** Install a supported Node.js LTS release, npm,

Git, and the current Power Apps CLI. Sign in and verify the intended account with `pa auth login` and `pa auth status` (Microsoft Learn: Power Apps CLI reference).

1. **[Code]** Install the locked project dependencies with `npm ci` (the repo

includes `package-lock.json`). If dependency changes are intentionally needed, use the team's package-update procedure; this handover does not change dependencies.

1. **[Code]** Check `power.config.json` locally and confirm the configured

environment is the intended development environment before any publishing command. The file contains a configured environment binding; this handover does not repeat its value.

1. **[Code]** Run `npm run build` to type-check and create the production web

build (`package.json` › `scripts.build`).

1. **[Verify in environment]** Run the Power Apps local host with `pa app run`

and sign in through the host. `npm run dev` starts the Vite development server, but does not replace the Power Apps host integration. The CLI command is documented in Microsoft's Power Apps CLI reference.

1. **[Verify in environment]** Confirm the expected role-gated landing page

appears and that Dataverse reads succeed for a test account. Do not use real payroll data for setup checks.

[Code] The available npm scripts are `dev`, `build`, `lint`, `preview`, `test`, and `test:watch` (`package.json` › `scripts`). The project guidance also recommends `pa app run`, `npm run build`, `npm run lint`, and `npm run preview` (`AGENTS.md` › Development Commands).

3. How data-source code is maintained

[Code] Registered Dataverse table data sources in `.power/schemas/appschemas/dataSourcesInfo.ts` are: `a365_departments`, `a365_employees`, `a365_hdmfcontributions`, `a365_payrollentries`, `a365_payrollperiods`, `a365_payrolltemplatelines`, `a365_payrolltemplates`, `a365_philhealthcontributions`, `a365_positions`,

`a365_ssscontributions`, `a365_taxbrackets`, `roles`, and `systemusers`. The same registration also contains the `whoami` Dataverse API operation. Generated models and services are under `src/generated/models/` and `src/generated/services/`.

[Code] Feature services use the shared SDK client in `src/lib/power/data-client.ts` › `dataClient`. Generated files are output from the Power Apps CLI and must not be edited by hand (`AGENTS.md` › Dataverse Rules; generated-file headers).

[Verify in environment] To add an existing Dataverse table, confirm its logical name in the intended environment first, then from the app root run:

```
npx --no-install pa app add data-source --connector dataverse --table <table-logical-name>
```

To refresh generated metadata for a registered source, use:

```
npx --no-install pa app refresh data-source --name <data-source-name>
```

These are current CLI command forms documented by Microsoft. The CLI writes generated schema/model/service artifacts, so review the diff and rebuild afterward. Microsoft Learn: CLI reference and Connect a code app to Dataverse.

4. Build and publish to Power Apps

[Code] For a development publish, the repository deployment runbook shows building first and then using `pa app push`. It also documents adding the app to a solution with `--solution-id` (`docs/deployment.md` › Step 1 and Updating an existing release). The general syntax is:

```
npm run build
npx --no-install pa app push --solution-id <solution-guid>
```

[Verify in environment] A push publishes to the active Power Apps environment/account. Before running it, check the local `power.config.json`, the active CLI account/environment, intended solution, app sharing, and release approval. Publishing changes the app in that environment; this document phase does not run a push. Microsoft documents `pa app push` as the publish command in its CLI reference.

5. Managed-solution release path

[Code] `docs/deployment.md` describes a managed-solution path: publish the Code App into the solution, export the solution as managed from development, select the intended target environment, import the solution, then perform environment-specific configuration and a smoke test. Its sample commands use `pac solution export`, `pac auth select`, and `pac solution import`.

[Verify in environment] Treat that runbook as a starting point, not proof of the current solution state. It contains a statement that the Code App was missing from the solution when written. Confirm current solution membership, version, target environment, connections, table data, role assignments, and deployment approvals in Power Platform before following it. No environment identifiers or URLs are reproduced here.

[Code] `power.config.json` currently has an `environmentId` binding and an empty `connectionReferences` object; `appName` and `solutionId` are null in the inspected file. The environment binding is sensitive operational context. Read the local file rather than copying an identifier from this guide. No environment variables are declared in this app's package scripts (`power.config.json`, `package.json` › `scripts`).

6. Useful commands

Purpose	Command	Evidence
Install dependencies	`npm ci`	**[Code]** `package-lock.json`
Vite development server	`npm run dev`	**[Code]** `package.json` › `scripts.dev`
Power Apps local host	`pa app run`	**[Verify in environment]** Requires working CLI sign-in and target environment; Microsoft Learn CLI reference.
TypeScript + production build	`npm run build`	**[Code]** `package.json` › `scripts.build`
Lint	`npm run lint`	**[Code]** `package.json` › `scripts.lint`
Unit tests	`npm test`	**[Code]** `package.json` › `scripts.test`
Preview built files	`npm run preview`	**[Code]** `package.json` › `scripts.preview`
Publish Code App	`pa app push`	**[Verify in environment]** Publishes to active Power Apps environment; CLI reference.

PART 2

Data model and action-to-table matrix

Map every Dataverse table and connect each user action to its data operations.

Sections	03 Data model · 04 User actions to tables
Status	Not started
Questions answered	What records and columns exist? Which actions read or change them?

PART 3

Business rules and security

Explain payroll rules, status transitions, access checks, and security boundaries.

Sections	05 Business rules · 06 Security and access
Status	Not started
Questions answered	How are payroll amounts and statuses determined? How is data access protected?

PART 4

Gaps, risks, and verification

Identify code risks and give the owner a practical environment verification checklist.

Sections	07 Risks, gaps, and verification
Status	Not started
Questions answered	What still needs attention or confirmation in Power Platform?